StockSense AI

Project Documentation & Development Roadmap

CS4063 — Natural Language Processing | Spring 2025

Mohammad Haider (23i-2558) • Hamd ul Haq (23i-0081) • Ayesha Ikram (23i-0109)

1. Executive Summary

StockSense AI is an intelligent, automated stock analysis platform that combines real-time financial news, social media sentiment, and historical price data to produce plain-English predictions about whether a stock is likely to go up, down, or hold steady. It functions as a financial analyst, a data scientist, and a news scanner working in concert — around the clock — so that investors never have to choose between reading the news and watching their charts.

The system is powered by FinBERT, a transformer-based NLP model trained specifically on financial text, paired with classical technical indicators (RSI, MACD, Moving Averages) and a Long Short-Term Memory (LSTM) neural network for time-series forecasting. All of these signals are fused together and served through a FastAPI REST interface, making the platform usable by individual investors, analysts, and even institutional teams.

At its core, StockSense AI addresses a fundamental gap in modern investing: the inability of any human to simultaneously absorb the volume, speed, and diversity of signals that financial markets generate every second. By automating this process end-to-end, the system converts raw, chaotic market data into a single, high-confidence directional forecast — delivered in under three seconds.

2. Problem Statement

2.1 Information Overload

On any given trading day, tens of thousands of news articles, earnings reports, analyst notes, Reddit threads, and tweets are published about publicly listed companies. Consider a single stock like NVIDIA (NVDA): on any earnings day, hundreds of news sources, dozens of Reddit posts, and thousands of tweets may be generated within hours. No human analyst — no matter how experienced — can read, process, and act on all of this simultaneously.

This overload creates a signal-to-noise problem. Somewhere inside all that data is a clear indication of market direction, but it is buried under irrelevant, duplicate, or misleading content. Investors are forced to rely on a tiny, manually selected slice of available information, which means their decisions are inherently incomplete.

2.2 Delayed Human Reaction vs. Real-Time Markets

Modern markets react within milliseconds to new information. Algorithmic trading systems operated by hedge funds and investment banks can process a news headline and execute a trade in under 50 milliseconds. A retail investor who reads the same headline, thinks about it, and decides to act may take 5 to 30 minutes — by which point the price has already moved, and the opportunity (or the warning) has passed.

Even for longer-horizon investors, delayed reactions cause real harm. A negative earnings surprise that drops a stock 8% is preceded by signals — declining social sentiment, reduced analyst estimates, cautious insider language in previous filings — that were available but not synthesized in time. StockSense AI is designed to close this timing gap.

2.3 Missed Connections Between News and Price Movement

Perhaps the most subtle but damaging problem is the failure to connect cause and effect across data sources. A tweet from a company CEO may be mildly positive in isolation, but when combined with a declining MACD indicator and a wave of cautious Reddit commentary, the combined signal may be strongly bearish. No single source tells the full story; only the fusion of multiple signals reveals the true picture.

Traditional tools force investors to use separate platforms for news, charts, and social media. The cognitive overhead of manually connecting these data streams means that most investors never see the complete, integrated picture. StockSense AI exists specifically to solve this integration problem.

3. Solution Overview

StockSense AI solves the three problems above through a fully automated, multi-signal intelligence pipeline. The core idea is simple: gather every relevant data signal, process each one with the best available analytical tool, and then combine them into a single, actionable prediction.

3.1 News Sentiment via NLP

The first signal layer is textual sentiment. Every article, Reddit post, and tweet related to a target stock is collected in real time and passed through FinBERT — a BERT-based model fine-tuned on financial language. Unlike keyword-matching approaches that simply look for words like 'growth' or 'loss', FinBERT understands context. It knows that 'the company failed to beat expectations' is bearish even though none of those individual words are negative in isolation.

Each text item receives a sentiment score (bullish, bearish, or neutral) along with a confidence probability. These scores are aggregated across sources to produce a composite sentiment index for the stock at any given moment.

3.2 Technical Indicators from Price Data

The second signal layer is quantitative. Using historical price and volume data from Yahoo Finance (via yfinance), the system computes the three most widely trusted technical indicators in trading: the Relative Strength Index (RSI), which measures momentum and identifies overbought or oversold conditions; the Moving Average Convergence Divergence (MACD), which captures trend direction and momentum shifts; and Moving Averages (both 20-day and 50-day), which indicate the medium and long-term price trajectory.

These indicators have been used by professional traders for decades and carry strong predictive value when combined with sentiment signals.

3.3 Machine Learning Prediction Engine

The third and final layer is the prediction engine, which fuses the sentiment index and the technical indicators into a single directional forecast. This is accomplished through an LSTM (Long Short-Term Memory) neural network — a type of recurrent neural network specifically designed for time-series data. LSTM networks are ideal here because they can learn patterns across time: they understand not just what the sentiment is today, but how it has been trending over the past several days.

The output of the prediction engine is a direction (Up, Down, or Stable) paired with a confidence percentage. This output is served through a FastAPI REST endpoint, making it accessible to dashboards, mobile apps, or any downstream application.

4. System Architecture (High-Level)

4.1 Data Sources Layer

The entry point of the system is a set of real-time data collectors that pull from three distinct categories of sources. Financial news is pulled from NewsAPI, which aggregates headlines and full article text from hundreds of reputable financial outlets. Social media sentiment is gathered from Reddit (specifically r/wallstreetbets and r/investing) via the PRAW library, and from Twitter/X via snscrape. Historical price and volume data is pulled from Yahoo Finance using the yfinance Python library.

Each data source is polled on a scheduled basis (e.g., every 15 minutes for news, every hour for social media) and stored in a structured database for further processing.

4.2 Processing Layer

The processing layer is responsible for cleaning raw data and transforming it into model-ready features. Text data goes through a preprocessing pipeline that strips HTML tags, removes URLs, normalizes Unicode characters, and eliminates duplicate entries. Numeric price data is passed through the technical indicator computation engine.

Both processing paths produce structured, timestamped records that can be joined by stock symbol and time window for downstream modeling.

4.3 Prediction Layer

The prediction layer houses the trained LSTM model. It receives a feature vector for each stock — combining the sentiment index, RSI value, MACD value, and moving average signals — and outputs a directional forecast with an associated confidence score. The model is retrained periodically on new data to prevent prediction drift.

4.4 Output and API Layer

The final layer is a FastAPI application that exposes the predictions as RESTful JSON endpoints. Clients can query a specific stock symbol and receive a structured response containing the predicted direction, confidence score, top sentiment signals, and key technical indicator values. An optional lightweight dashboard (built with Streamlit or a similar tool) provides a visual interface for non-technical users.

5. Detailed Pipeline & Development Roadmap

This section provides a phase-by-phase construction guide for building StockSense AI from scratch. Each phase covers what to build, why it is needed, which tools and libraries to use, what the expected output looks like, and what challenges to anticipate.

Phase 1: Data Collection

What needs to be done

You need to build automated collectors that pull data from three source types: financial news APIs, social media platforms (Reddit and Twitter/X), and stock price APIs. Each collector should run on a schedule and persist its output to a database.

Why it is needed

All downstream analysis depends on having a consistent, up-to-date stream of raw data. Without a reliable collection layer, the rest of the pipeline has nothing to work with. The diversity of sources is also critical — relying on only news or only social media produces a one-dimensional view of market sentiment.

Tools and libraries

- NewsAPI (newsapi.org) — provides a Python client and REST API for pulling headlines and article bodies from 80,000+ news sources. Free tier allows 100 requests/day.
- PRAW (Python Reddit API Wrapper) — official Reddit API library. Register a Reddit app at reddit.com/prefs/apps to get credentials. Supports fetching top posts and comments from any subreddit.
- snscrape — scrapes Twitter/X without requiring official API credentials. Install via pip: pip install snscrape. Supports keyword and cashtag searches (e.g., $AAPL).
- yfinance — Yahoo Finance wrapper. Fetches historical OHLCV (Open, High, Low, Close, Volume) data. Call yf.download('AAPL', period='6mo', interval='1d') to get six months of daily data.
- SQLite or PostgreSQL — for local development, SQLite (built into Python) is sufficient. For production, PostgreSQL provides better concurrency and indexing. Use SQLAlchemy as the ORM layer.
- APScheduler or cron — to schedule periodic data fetches. APScheduler can be embedded directly in the Python application. cron jobs are suitable for production Linux deployments.

How to implement — step by step

First, set up your project structure with separate modules for each data source. Create a news_collector.py, reddit_collector.py, twitter_collector.py, and price_collector.py. Each module should expose a fetch() function that returns a list of standardized records.

For NewsAPI, initialize the client with your API key and call newsapi.get_everything(q='AAPL', language='en', sort_by='publishedAt'). Map each article to a dictionary with fields: source_id, title, description, published_at, url, and raw_text.

For PRAW, authenticate with your app credentials and call reddit.subreddit('wallstreetbets+investing').search('AAPL', sort='new', time_filter='day'). Extract the title, selftext, score, and created_utc from each submission object.

For snscrape, run it as a subprocess: subprocess.run(['snscrape', '--jsonl', '--max-results', '100', 'twitter-search', '$AAPL since:2024-01-01']), then parse the JSON output line by line.

For yfinance, call yf.download(ticker, period='1y', interval='1d') and save the resulting DataFrame directly to CSV or a database table. Schedule this to run once per day after market close (4:30 PM EST).

Expected output

A database with at least four tables: news_articles (id, symbol, title, body, source, published_at), reddit_posts (id, symbol, title, body, score, created_at), tweets (id, symbol, text, created_at), and price_data (id, symbol, date, open, high, low, close, volume).

Challenges and considerations

- Rate limits: NewsAPI free tier limits requests per day. Cache fetched results aggressively and avoid re-fetching articles you already have. Use the url field as a deduplication key.
- Twitter/X access: snscrape may break if Twitter changes its frontend. Consider using the official Twitter API v2 (free Basic tier) as a fallback.
- Timezone handling: Always store timestamps in UTC. Convert to local time only at the display layer to avoid confusion when joining news events with price data.
- Missing data: yfinance may return gaps for weekends and holidays. Fill these with the previous close value (forward-fill) to maintain a continuous time series.

Phase 2: Data Preprocessing

What needs to be done

Raw text from news and social media is messy. It contains HTML tags, URLs, emojis, special characters, boilerplate footers, and duplicate entries. This phase cleans and normalizes all text data so it is ready for NLP processing.

Why it is needed

Language models like FinBERT perform best on clean, well-formed text that resembles the financial documents they were trained on. Noise in the input (e.g., a URL or a sequence of emojis) can confuse the tokenizer and degrade sentiment accuracy. Preprocessing is a non-negotiable step between raw data collection and model inference.

Tools and libraries

- re (Python built-in) — for regex-based cleaning of URLs, HTML tags, and special characters.
- BeautifulSoup4 — for parsing and stripping HTML tags from article bodies.
- ftfy — 'Fixes Text For You'. Handles Unicode normalization and encoding artifacts (e.g., converting â€™ back to an apostrophe).
- pandas — for working with DataFrames, deduplication, and batch processing.
- langdetect — for filtering out non-English content. Call detect(text) and skip records where the result is not 'en'.

How to implement — step by step

Create a preprocessing.py module with a clean_text(text) function. Apply the following steps in order:

- Strip HTML: Use BeautifulSoup(text, 'html.parser').get_text() to remove all HTML markup.

- Remove URLs: Apply re.sub(r'http\S+', '', text) to remove hyperlinks.
- Remove ticker cashtags and usernames: Apply re.sub(r'\$[A-Z]+|@\w+', '', text) unless you want to keep tickers as features.
- Normalize whitespace: Apply re.sub(r'\s+', ' ', text).strip() to collapse multiple spaces and remove leading/trailing whitespace.
- Fix encoding: Pass text through ftfy.fix_text(text) to resolve Unicode issues.
- Language filter: Use langdetect to skip non-English records.
- Deduplicate: Use pandas drop_duplicates on the cleaned text column to remove near-identical articles.
- Length filter: Discard records shorter than 20 characters — they are typically not meaningful for sentiment analysis.

Expected output

A cleaned DataFrame or database table where each row contains a stock symbol, cleaned text, source type (news/reddit/twitter), and timestamp. The text should be human-readable with no HTML, no URLs, and no encoding artifacts.

Challenges and considerations

- Boilerplate footers: Many news articles end with disclaimers like 'This article was generated by a news agency.' These add no sentiment value. Use a rule-based filter to truncate text at common boilerplate markers.
- Sarcasm and irony: Phrases like 'great job, Tim Cook' may be sarcastic. FinBERT handles this better than keyword matching, but no NLP model is immune. Treat sarcastic posts as noise that averages out across many records.
- Emoji sentiment: Emojis like 🚀 and 📈 carry strong bullish sentiment on Reddit. You may choose to convert them to text equivalents (e.g., 'rocket' for 🚀) before passing them to FinBERT, rather than stripping them outright.

Phase 3: Sentiment Analysis with FinBERT

What needs to be done

Apply FinBERT to each cleaned text record to produce a sentiment classification (Positive/Bullish, Negative/Bearish, or Neutral) and a confidence score. Aggregate these scores per stock symbol and time window to produce a composite sentiment index.

Why it is needed

Sentiment is the primary signal that distinguishes StockSense AI from a simple technical trading system. Understanding what the market is saying — not just what prices are doing — allows the system to anticipate moves before they appear in price data. Earnings surprises, CEO comments, and regulatory news all show up in language before they show up in charts.

Tools and libraries

- transformers (HuggingFace) — provides the pre-trained FinBERT model. Install with pip install transformers torch.

- FinBERT model ID: ProsusAI/finbert — load with pipeline('text-classification', model='ProsusAI/finbert').
- torch — PyTorch backend for transformer inference. GPU not required for batch inference on a laptop.
- pandas — for batch processing DataFrames.

How to implement — step by step

Create a sentiment.py module. Load the FinBERT pipeline once at module load time to avoid reloading the model for each batch. Then write a score_texts(texts) function that accepts a list of strings and returns a list of (label, score) tuples.

```
from transformers import pipeline
finbert = pipeline('text-classification', model='ProsusAI/finbert',
top_k=None)

def score_texts(texts, batch_size=32):
    results = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        preds = finbert(batch, truncation=True, max_length=512)
        results.extend(preds)
    return results
```

FinBERT outputs three scores (one per label). Map 'positive' → +1, 'negative' → -1, 'neutral' → 0. Multiply each by its confidence score to produce a weighted sentiment value. Sum these values across all records for a given stock in a given time window (e.g., the past 6 hours) to produce a composite sentiment index.

For example, if you have 40 news articles for AAPL in the past 6 hours, compute: sentiment_index = sum(weight * direction) / count. A value near +1 indicates strongly bullish coverage; near -1 indicates strongly bearish.

Expected output

A sentiment_scores table (or DataFrame) with columns: symbol, window_start, window_end, sentiment_index, record_count. This table feeds directly into the feature engineering phase.

Challenges and considerations

- Text length limit: FinBERT (like all BERT-based models) has a maximum input length of 512 tokens. For long articles, truncate to the first 512 tokens (which is usually the most information-dense part) or split into paragraphs and average the scores.
- Inference speed: FinBERT is a 110M-parameter model. On CPU, expect roughly 2–5 seconds per batch of 32 texts. For real-time requirements, run batch inference in a background worker and cache results, rather than running inference in the request path.
- Label calibration: FinBERT's confidence scores are not perfectly calibrated — a score of 0.9 does not necessarily mean 90% certainty. Treat scores as relative indicators rather than absolute probabilities.
- Source weighting: Not all sources are equally reliable. Consider applying a trust multiplier — e.g., giving Reuters articles 1.5x weight versus anonymous Reddit posts at 0.7x weight — before computing the composite index.

Phase 4: Stock Data & Technical Indicators

What needs to be done

Fetch historical OHLCV price data for each target stock and compute the three core technical indicators: RSI, MACD, and Moving Averages. These indicators describe the stock's price momentum, trend direction, and relative strength — independent of news sentiment.

Why it is needed

Technical indicators provide the quantitative counterpart to qualitative sentiment. A stock may have positive news coverage but a declining MACD and an RSI above 70 (indicating it is overbought). In such cases, the technical signal should moderate the bullish sentiment. The fusion of both signal types produces predictions that are more reliable than either alone.

Tools and libraries

- yfinance — fetches historical price data from Yahoo Finance. Free, no API key required.
- pandas — for time-series manipulation and rolling window calculations.
- TA-Lib — comprehensive technical analysis library with C bindings for speed. Install with pip install TA-Lib (requires the TA-Lib C library; on Ubuntu: apt install libta-lib-dev). Alternatively, use the pandas-ta library which is pure Python and easier to install.
- pandas-ta — easier alternative to TA-Lib: pip install pandas-ta. Supports 130+ indicators including RSI, MACD, and all moving average variants.

How to implement — step by step

Create a technical_indicators.py module. Fetch six months of daily data for each target stock using yfinance. Then compute indicators using pandas-ta:

```
import yfinance as yf
import pandas_ta as ta

def compute_indicators(symbol):
    df = yf.download(symbol, period='6mo', interval='1d',
auto_adjust=True)
    df.ta.rsi(length=14, append=True)           # adds RSI_14 column
    df.ta.macd(fast=12, slow=26, signal=9, append=True)  # adds MACD,
MACDh, MACDs
    df.ta.sma(length=20, append=True)           # adds SMA_20
    df.ta.sma(length=50, append=True)           # adds SMA_50
    return df.dropna()
```

Interpret each indicator as follows: RSI above 70 → overbought (bearish signal); RSI below 30 → oversold (bullish signal); RSI between 30 and 70 → neutral. MACD line crossing above the signal line → bullish; crossing below → bearish. Price above SMA_20 and SMA_50 → uptrend; below → downtrend.

Encode these interpretations as numeric features: RSI_signal (+1, 0, -1), MACD_signal (+1, 0, -1), trend_signal (+1, 0, -1). These three values, combined with the raw RSI and MACD numbers, form the technical feature vector.

Expected output

A indicators table or DataFrame with columns: symbol, date, close, rsi_14, macd, macd_signal_line, sma_20, sma_50, rsi_signal, macd_signal, trend_signal.

Challenges and considerations

- Warm-up period: RSI requires at least 14 data points to compute; MACD requires at least 26. Always fetch more historical data than you need (6 months is safe) and drop the first 50 rows to ensure all indicators are fully populated.
- Intraday vs. daily: The system uses daily data for the prediction model, but you may want to compute intraday indicators (15-minute or hourly) for more timely signals. This requires a paid yfinance tier or an alternative data provider like Polygon.io.
- Survivorship bias: Test stocks that have been continuously listed for the entire backtest period. Stocks that were delisted during the period introduce bias.

Phase 5: Feature Engineering

What needs to be done

Combine the sentiment index (from Phase 3) and the technical indicators (from Phase 4) into a single, unified feature matrix. Each row in this matrix represents one stock at one point in time, and it will be the input to the prediction model.

Why it is needed

Machine learning models require structured, numeric input. Feature engineering is the process of translating your domain knowledge — which signals matter, how they interact, what time windows are relevant — into the specific numeric format the model expects. Poor feature engineering is the most common reason prediction models underperform.

Tools and libraries

- pandas — for joining DataFrames on symbol and date keys.
- scikit-learn — for feature scaling (StandardScaler or MinMaxScaler). LSTMs are sensitive to feature magnitude; all features should be normalized to a similar range.
- numpy — for constructing the 3D arrays that LSTM requires.

How to implement — step by step

Join the sentiment and indicator DataFrames on (symbol, date). Handle the alignment carefully: sentiment is computed over a rolling 6-hour window, but your model likely operates on daily data. Aggregate sentiment to daily resolution by taking the mean sentiment_index for each stock-date pair.

After joining, your feature matrix should have columns: symbol, date, sentiment_index, record_count, rsi_14, macd, macd_signal_line, sma_20, sma_50, rsi_signal, macd_signal, trend_signal, close, and the target label (next_day_direction: +1 for up, -1 for down, 0 for flat).

Define next_day_direction as: +1 if close(t+1) > close(t) * 1.005 (up more than 0.5%), -1 if close(t+1) < close(t) * 0.995, and 0 otherwise. The 0.5% threshold prevents mislabeling trivial fluctuations as meaningful moves.

Scale all numeric features using MinMaxScaler with range (-1, 1). Fit the scaler on training data only; transform validation and test data using the fitted scaler to prevent data leakage.

Finally, reshape the data into 3D arrays for LSTM: (samples, timesteps, features). Use a lookback window of 10 days — each sample is a sequence of 10 consecutive days of features, and the label is the direction on day 11.

Expected output

NumPy arrays: X_train (shape: [n_train, 10, n_features]), X_test (shape: [n_test, 10, n_features]), y_train (shape: [n_train]), y_test (shape: [n_test]). A saved scaler object (joblib.dump(scaler, 'scaler.pkl')) for use during inference.

Challenges and considerations

- Missing sentiment data: Some days may have no news coverage for a given stock. Fill sentiment_index with 0 (neutral) and record_count with 0 to indicate the absence of signal.
- Class imbalance: In most datasets, 'stable' (0) days outnumber 'up' and 'down' days. Use class_weight='balanced' in your model or oversample minority classes with SMOTE to prevent the model from predicting 'stable' for everything.
- Feature correlation: RSI and MACD are correlated. Consider using Principal Component Analysis (PCA) to reduce dimensionality, though with only 10–12 features this is usually not necessary.

Phase 6: Prediction Model

What needs to be done

Train a machine learning model on the feature matrix to predict the next-day price direction. The primary recommended architecture is an LSTM network. Evaluate the model and save it for inference.

Why LSTM is the right choice

LSTM (Long Short-Term Memory) is a type of recurrent neural network that is specifically designed to learn patterns in sequential data. Unlike a standard neural network, which treats each day's features as independent, an LSTM processes the 10-day lookback sequence and learns how patterns evolve over time — for example, that a three-day decline in sentiment followed by a bullish MACD crossover tends to precede a price recovery. This temporal reasoning is exactly what stock prediction requires.

Tools and libraries

- TensorFlow / Keras — the most accessible deep learning framework for LSTMs. Install with pip install tensorflow.
- scikit-learn — for baseline models (Logistic Regression, Random Forest) to benchmark against LSTM.
- joblib — for saving and loading trained model artifacts.
- matplotlib — for plotting training curves and confusion matrices.

How to implement — step by step

Start by training a Logistic Regression baseline. This takes seconds and establishes the minimum accuracy floor your LSTM needs to beat. A baseline accuracy of 50–55% is typical for pure technical indicator models on daily data.

Then build and train the LSTM model using Keras:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout,
BatchNormalization

model = Sequential([
    LSTM(64, input_shape=(10, n_features), return_sequences=True),
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dropout(0.2),
    BatchNormalization(),
    Dense(16, activation='relu'),
    Dense(3, activation='softmax')   # 3 classes: up, down, stable
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=50, batch_size=32,
          validation_split=0.1, callbacks=[early_stopping])
```

Use EarlyStopping with patience=10 to prevent overfitting. Monitor val_loss, not val_accuracy, as the stopping criterion. Save the best model with ModelCheckpoint.

After training, evaluate on the held-out test set. Report accuracy, precision, recall, and F1-score per class. If direction accuracy (the fraction of up/down calls that are correct, excluding stable) is above 60%, the model is considered production-ready per the project's success criteria.

Expected output

A saved model file (model.h5 or SavedModel directory), a confusion matrix, and a classification report. The model should achieve at least 60% directional accuracy on the test set.

Challenges and considerations

- Overfitting: With limited financial data (a few thousand samples for a single stock), LSTMs can easily overfit. Use Dropout (0.2–0.3) and EarlyStopping. If overfitting persists, reduce model complexity or use an L2 regularizer.
- Market regime changes: A model trained on bull market data will perform poorly in a bear market. Retrain or fine-tune the model periodically (e.g., monthly) on recent data using a rolling window approach.
- Look-ahead bias: Never include any feature derived from the target day (day t+1) in the input features. This is the most critical and common error in financial ML. Double-check your feature engineering for any accidental data leakage.
- Multi-stock vs. per-stock: Training one model per stock captures stock-specific patterns but requires more data. Training a single model across all stocks pools data but may not capture idiosyncratic behavior. Start with one model per stock; generalize later if data is scarce.

Phase 7: Backend & REST API

What needs to be done

Build a FastAPI application that wraps the prediction pipeline into a set of RESTful endpoints. When a client sends a stock symbol, the API should trigger data collection, preprocessing, and model inference, and return a structured JSON prediction.

Why it is needed

A raw Python model is not usable by a web dashboard, a mobile app, or any external system. The API layer is what transforms your model from a script into a service. FastAPI is the ideal choice because it is fast (based on Starlette and Pydantic), generates automatic OpenAPI documentation, and supports async operations natively — which is important for non-blocking I/O when calling external APIs.

Tools and libraries

- FastAPI — install with pip install fastapi uvicorn.
- Pydantic — used by FastAPI for request and response schema validation.
- uvicorn — ASGI server for running FastAPI in production.
- apscheduler — for running background data collection tasks within the API process.

How to implement — step by step

Create a main.py file with three endpoints:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title='StockSense AI', version='1.0')

class PredictionResponse(BaseModel):
    symbol: str
    direction: str          # 'UP', 'DOWN', or 'STABLE'
    confidence: float       # 0.0 to 1.0
    sentiment_index: float  # -1.0 to 1.0
    rsi: float
    macd: float
    generated_at: str       # ISO 8601 timestamp

@app.get('/predict/{symbol}', response_model=PredictionResponse)
async def predict(symbol: str):
    # 1. Fetch latest data for symbol
    # 2. Preprocess and compute features
    # 3. Run model inference
    # 4. Return structured response
    ...
```

Add a /health endpoint that returns HTTP 200 with status: ok for load balancer health checks. Add a /sentiment/{symbol} endpoint that returns the raw sentiment breakdown (bullish %, bearish %, neutral %) without a price prediction, for users who want to inspect the NLP output.

Run the server with: uvicorn main:app --host 0.0.0.0 --port 8000 --reload (development) or uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4 (production).

Expected output

A running API server accessible at http://localhost:8000. Calling GET /predict/AAPL returns a JSON object like: { 'symbol': 'AAPL', 'direction': 'UP', 'confidence': 0.76, 'sentiment_index': 0.42, 'rsi': 58.3, 'macd': 1.24, 'generated_at': '2025-04-10T14:30:00Z' }. The full API schema is auto-documented at http://localhost:8000/docs.

Challenges and considerations

- Latency: The 3-second response time target requires that the most expensive operations (FinBERT inference, model prediction) are pre-computed and cached. Implement a prediction cache with a 15-minute TTL so that repeated queries for the same symbol return instantly.
- Error handling: Wrap all data fetch calls in try-except blocks and return informative HTTP error codes. If NewsAPI returns a 429 (rate limit), return HTTP 503 with a Retry-After header.
- Security: Add API key authentication using FastAPI's HTTPBearer dependency if the API is to be publicly accessible. Without authentication, anyone can hit your endpoints and exhaust your rate limits.

Phase 8: Integration & Orchestration

What needs to be done

Connect all components — data collection, preprocessing, sentiment analysis, technical indicators, and the prediction model — into a single, coherent pipeline that can be triggered either on-demand (by an API call) or on a schedule (background refresh).

Why it is needed

Having each component work in isolation is not sufficient. The integration layer ensures that data flows correctly from one stage to the next, that errors in one stage are handled gracefully, and that the entire pipeline can be run end-to-end without manual intervention.

Tools and libraries

- LangChain (optional) — if you want to build an agentic layer that can answer natural language questions about stocks (e.g., 'Why is NVDA predicted to go up?'), LangChain provides the orchestration primitives. It can call your prediction API, interpret the results, and generate a plain-English explanation. This is an optional enhancement, not a core requirement.
- Celery + Redis — for robust task queue management in production. Celery workers can process data collection and inference tasks asynchronously, with Redis as the message broker. For a course project, APScheduler is simpler.

- Python dataclasses or Pydantic models — for defining a standard PipelineResult schema that all pipeline stages produce and consume.

How to implement — step by step

Create a pipeline.py module with a run_pipeline(symbol) function that calls each stage in sequence: fetch_data(symbol) → preprocess(data) → compute_sentiment(texts) → compute_indicators(symbol) → engineer_features(sentiment, indicators) → predict(features). Each function should return a typed result object, and the pipeline function should handle exceptions from each stage with appropriate fallbacks.

If LangChain integration is desired, create a StockAnalysisAgent that uses a ReAct (Reasoning + Acting) prompt pattern. The agent is given the prediction output and asked to explain it in plain English, citing the top sentiment headlines and the most significant technical indicator signals as evidence.

Expected output

A single pipeline.run_pipeline('AAPL') call that executes the entire workflow end-to-end and returns a complete prediction with supporting evidence. All intermediate results are logged for debugging.

Challenges and considerations

- Partial failures: If the Reddit API is temporarily unavailable, the pipeline should still run using only news and technical data. Build fallback logic that degrades gracefully rather than failing entirely.
- Pipeline idempotency: Running the pipeline twice for the same symbol and time window should produce the same result. Cache intermediate results (preprocessed text, computed indicators) with a TTL keyed on symbol and date.

Phase 9: Frontend / Output Layer

What needs to be done

Build a user-facing interface that displays predictions in a clear, actionable format. For a course project, a simple Streamlit dashboard or a CLI tool is sufficient. The goal is to demonstrate that the predictions are accessible to a non-technical user.

Why it is needed

Raw JSON from an API endpoint is not the final deliverable — it is an intermediate artifact. The output layer translates technical predictions into human-friendly visualizations and plain-English summaries that a retail investor can act on immediately.

Tools and libraries

- Streamlit — the fastest way to build a Python data dashboard. Install with pip install streamlit. A complete dashboard can be built in under 200 lines of Python.
- Plotly — for interactive stock charts. Integrates natively with Streamlit.
- requests — for calling the FastAPI backend from the Streamlit frontend.

How to implement — step by step

Create a dashboard.py file. Use st.selectbox to let the user choose a stock symbol from the five targets (AAPL, TSLA, AMZN, MSFT, NVDA). On selection, call your FastAPI endpoint with requests.get(f'http://localhost:8000/predict/{symbol}') and display the result.

Display the prediction direction as a large, color-coded badge: green arrow for UP, red arrow for DOWN, grey dash for STABLE. Show the confidence score as a Streamlit progress bar. Display a Plotly candlestick chart of the last 30 days of price data with the moving averages overlaid. Show the top 5 most recent sentiment-influencing headlines as a table.

For a CLI alternative, use Python's click library: @click.command() with a --symbol option. Print the prediction in formatted text to the terminal.

Expected output

A running Streamlit app (streamlit run dashboard.py) that shows a real-time prediction dashboard. The dashboard should update automatically when the user selects a different stock symbol.

Phase 10: Testing & Evaluation

What needs to be done

Systematically evaluate the performance of every component in the system: sentiment accuracy, prediction accuracy, API response time, and overall system reliability. Document the results.

Why it is needed

A model that performs well on training data but poorly on unseen data is not useful. Rigorous evaluation — on held-out test data that the model has never seen — is the only way to produce honest estimates of real-world performance. This section is also what separates a serious engineering project from a demo.

Tools and libraries

- scikit-learn metrics — classification_report, confusion_matrix, accuracy_score for model evaluation.
- pytest — for writing unit tests for each pipeline component.
- locust or httpx — for API load testing (measure response time under concurrent requests).
- matplotlib / seaborn — for plotting confusion matrices and accuracy curves.

How to evaluate — step by step

Sentiment validation: Manually label 200 financial headlines as positive, negative, or neutral. Run them through FinBERT and compare the model's labels to your manual labels. Report precision, recall, and F1-score per class. A well-calibrated FinBERT should achieve 85–90% accuracy on financial text.

Prediction accuracy: Use a walk-forward validation strategy. Train on data from month 1 through month 10, test on month 11. Then train on months 1–11, test on month 12, and so on. Average the accuracy across all test windows. This simulates how the model would perform if deployed live.

API performance: Use httpx to send 50 concurrent requests to the /predict/AAPL endpoint. Measure the 50th, 95th, and 99th percentile response times. The p95 should be under 3 seconds.

End-to-end live test: Run the system on the five target stocks (AAPL, TSLA, AMZN, MSFT, NVDA) for five consecutive trading days. Record each prediction and compare it to the actual next-day closing direction. This is your live demo accuracy.

Expected output

A testing report containing: sentiment validation accuracy (target: ≥85%), directional prediction accuracy (target: ≥60%), API p95 response time (target: <3 seconds), and a 5-day live demo accuracy table.

Challenges and considerations

- Ground truth labeling: For sentiment validation, the ground truth is your manual annotation, which is subjective. Have at least two team members independently label the same 200 headlines and measure inter-annotator agreement (Cohen's kappa). Only use headlines where both annotators agree as the evaluation set.
- Benchmark comparison: Your LSTM model should be compared not just to a random baseline (33% for 3-class) but to a strong baseline like a Random Forest trained on the same features. If Random Forest performs comparably to LSTM, the complexity of the LSTM may not be justified.

6. Tools & Technologies Summary

Component	Tool / Library	Purpose
News Data	NewsAPI	Fetches financial headlines and article text in real time
Reddit Data	PRAW	Pulls posts and comments from financial subreddits
Twitter Data	snscrape	Scrapes tweets by cashtag without requiring API keys
Stock Price Data	yfinance	Downloads historical OHLCV data from Yahoo Finance
Text Preprocessing	BeautifulSoup4, ftfy, re	Cleans HTML, fixes encoding, removes noise
Sentiment Analysis	FinBERT (HuggingFace)	Context-aware financial sentiment classification
Technical Indicators	pandas-ta	Computes RSI, MACD, and Moving Averages

Component	Tool / Library	Purpose
Feature Engineering	pandas, scikit-learn	Joins signals and scales features for ML input
Prediction Model	TensorFlow / Keras	LSTM neural network for time-series direction forecasting
Model Serialization	joblib / h5	Saves trained model and scaler for production inference
Backend API	FastAPI + uvicorn	REST API serving predictions as structured JSON
Task Scheduling	APScheduler	Runs background data collection jobs on a schedule
Orchestration	LangChain (optional)	Agentic layer for natural language explanations
Containerization	Docker	Packages the full application for consistent deployment
Frontend Dashboard	Streamlit + Plotly	Interactive prediction dashboard for end users
Database	PostgreSQL / SQLite	Persistent storage for raw data and processed features
Testing	pytest, scikit-learn metrics	Unit testing and model evaluation

7. Success Metrics

7.1 Directional Prediction Accuracy (Target: ≥ 60%)

The primary metric for the prediction model is directional accuracy — the fraction of Up/Down predictions that match the actual next-day price movement. A 60% threshold is the target. This may seem modest, but in financial markets, even a sustained 55% accuracy rate is commercially significant. Random guessing yields 50% (binary), and professional quantitative funds often operate on edges of 52–55%. A 60% target is therefore both realistic and meaningful.

Accuracy is measured using walk-forward validation on historical data and confirmed with a 5-day live demo on the five target stocks.

7.2 Sentiment Validation Accuracy (Target: ≥ 85%)

Before the sentiment signal can be trusted as a model input, it must be independently validated. A set of 200 financial headlines is manually labeled by team members and compared against FinBERT's outputs. The target is ≥85% agreement. This ensures that the NLP component is

actually interpreting financial language correctly, rather than producing random or systematically biased labels.

7.3 API Response Time (Target: < 3 seconds)

For the system to be useful in practice, predictions must be returned quickly. A 3-second end-to-end latency target is set for the /predict/{symbol} endpoint. This is achievable with a prediction cache that stores recent results and returns them instantly on repeated queries. The 3-second target applies to cold requests (uncached); cached requests should return in under 200 milliseconds.

7.4 Live Demo on Five Target Stocks

The final validation is a live demo covering AAPL (Apple), TSLA (Tesla), AMZN (Amazon), MSFT (Microsoft), and NVDA (NVIDIA). These five stocks were selected because they have high news coverage, active social media communities, and strong liquidity — ideal conditions for the multi-source approach. The live demo demonstrates that the system works on real, current market data and is not overfitted to historical test data.

8. Challenges & Risks

8.1 Noisy and Unreliable Data

Financial social media — especially Reddit and Twitter — contains a significant proportion of noise, humor, sarcasm, and outright misinformation. A viral meme about a stock may generate hundreds of posts with strong positive language that does not reflect genuine investor sentiment. FinBERT mitigates this by understanding context, but it is not immune to coordinated manipulation or low-quality content. Source weighting and minimum engagement thresholds (e.g., only include Reddit posts with at least 10 upvotes) help, but noisy data will always be a challenge.

8.2 Market Unpredictability

Financial markets are affected by macroeconomic events, geopolitical developments, regulatory changes, and black swan events that no model can predict from news and technical indicators alone. A Federal Reserve interest rate announcement, a sudden regulatory action, or a natural disaster can move markets dramatically in ways that have no historical precedent in the training data. The system should be presented honestly as a decision-support tool, not a guaranteed predictor.

8.3 Model Accuracy Limitations

The 60% accuracy target is a minimum threshold, not a guarantee. LSTM models on financial data are prone to overfitting on training data and underperforming in live conditions. The walk-forward validation strategy mitigates this, but there will always be periods where the model's predictions are systematically wrong — for example, during earnings season or market crashes. Regular retraining and continuous evaluation are essential.

8.4 API Rate Limits and Data Delays

Free-tier APIs (NewsAPI, Reddit) impose rate limits that may delay data collection during high-activity periods. A major earnings announcement may generate thousands of articles and posts within minutes, causing rate limit errors. The system must implement exponential backoff, request queuing, and graceful degradation (using cached data when fresh data is unavailable) to remain operational under these conditions.

8.5 FinBERT Inference Latency

Running FinBERT on CPU takes 2–5 seconds per batch of 32 texts. In a high-volume scenario with hundreds of articles per request, this becomes a bottleneck. The solution is to pre-compute and cache sentiment scores in a background job, so that API requests retrieve cached results rather than triggering on-demand FinBERT inference.

9. Future Improvements

9.1 Real-Time Streaming Pipeline

The current architecture is batch-oriented — data is collected and processed on a schedule. A natural evolution is to replace the scheduler with a real-time streaming pipeline using Apache Kafka or AWS Kinesis. Incoming news articles and tweets would be processed as they arrive, rather than in 15-minute batches. This would reduce end-to-end latency from minutes to seconds, enabling genuine intraday predictions.

9.2 More Advanced Prediction Models

Several architectural improvements to the prediction model are worth exploring. Temporal Fusion Transformers (TFTs) are state-of-the-art for multi-horizon time-series forecasting and have outperformed LSTMs on financial benchmarks. Ensemble methods — combining LSTM, Random Forest, and a Gradient Boosting model — can also improve robustness. Additionally, incorporating earnings call transcripts and SEC filings (available via the SEC EDGAR API) would add a fundamentals layer to the current sentiment-plus-technicals approach.

9.3 Portfolio-Level Recommendations

The current system predicts direction for individual stocks. A natural extension is to aggregate predictions across a portfolio and recommend allocation changes: increase position in strongly bullish stocks, reduce position in bearish ones, hold neutral stocks. This would require a portfolio optimization layer (e.g., using scipy.optimize or the pyportfolioopt library) that translates individual predictions into buy/sell/hold recommendations with position sizing.

9.4 User Personalization

Different investors have different risk tolerances, investment horizons, and areas of interest. A personalization layer would allow users to specify their stock watchlist, their confidence threshold (e.g., only show predictions with confidence > 75%), and their preferred data sources. A user who distrusts social media could configure the system to use only news and technical data. This would transform StockSense AI from a generic tool into a personalized investment assistant.

9.5 Backtesting Engine

A rigorous backtesting engine would allow users to simulate trading strategies based on StockSense AI predictions over historical data. The engine would track simulated portfolio value over time, compute performance metrics (Sharpe ratio, maximum drawdown, annualized return), and compare against a benchmark (e.g., buy-and-hold S&P 500). This would provide quantitative evidence of the system's investment value beyond accuracy percentages.

9.6 Alternative Language Model Integration

As large language models (LLMs) become more capable and cheaper to run, they could replace or supplement FinBERT for sentiment analysis. A GPT-4 or Claude-based analysis layer could not only classify sentiment but also extract key entities (companies mentioned, executives quoted), identify causal claims ('NVDA rose because of strong data center demand'), and generate structured summaries of complex documents like earnings call transcripts. This would significantly enrich the feature set available to the prediction model.